

webdriver — browser automation with Babet

webdriver.lua is a **W3C WebDriver** client for Babet 2.9.0 or newer. It drives Firefox, Chrome and Chromium through geckodriver or chromedriver.

Table of contents

- [Installation](#)
- [Starting a new project](#)
- [Creating a session](#)
- [Start options](#)
- [Driver lifecycle](#)
- [Navigation](#)
- [Locating elements and waiting](#)
- [Elements](#)
- [JavaScript](#)
- [Keyboard and mouse actions](#)
- [Windows, tabs and frames](#)
- [Alerts, cookies and timeouts](#)
- [Screenshots](#)
- [Automatic driver management](#)
- [Workers and channels](#)
- [Error contract](#)
- [Limitations](#)

Installation

The files webdriver.lua, driver_manager.lua and, when needed, webdriver_worker.lua must be reachable through package.path.

```
local webdriver = require("webdriver")
```

The library requires:

- Babet 2.9.0+;
- Linux;
- Firefox, Chrome or Chromium;
- geckodriver or chromedriver, either installed or downloadable.

It does not require tar, unzip, curl, base64 or an external shell.

Starting a new project

Direct project: required files

To drive a browser from the main script, copy only:

```
my-project/  
├─ webdriver.lua  
├─ driver_manager.lua  
└─ main.lua
```

webdriver.lua loads driver_manager.lua internally. Both modules are therefore required even when geckodriver or chromedriver is already installed.

When the modules are next to main.lua, loading is direct:

```
local webdriver = require("webdriver")
```

Project using a worker

To host the WebDriver session inside a worker and communicate through channels, add the proxy module:

```
my-project/
├── webdriver.lua
├── driver_manager.lua
├── webdriver_worker.lua
└── main.lua
```

The main script then loads:

```
local webdriver_worker = require("webdriver_worker")
```

tools/check_env.lua is an optional diagnostic tool. The tests/, examples/ and docs/ directories, as well as build_docs.sh and the run_*.sh scripts, are not required in an application project.

Location of the Babet executable

The repository's bin/ directory is only a convenient convention. Babet may be installed anywhere.

Project-local executable:

```
my-project/
├── bin/babet
├── webdriver.lua
├── driver_manager.lua
└── main.lua
```

```
./bin/babet main.lua
```

Babet installed in PATH:

```
babet main.lua
```

A shebang also allows:

```
#!/usr/bin/env babet
```

```
chmod +x main.lua
./main.lua
```

Babet at an arbitrary path:

```
/opt/babet/bin/babet main.lua
```

Inside the babet-webdriver repository, run_tests.sh, run_smoke.sh and run_worker_smoke.sh locate the executable in this order:

1. the path provided through BABET;
2. bin/babet;
3. a babet command available in PATH.

Example:

```
BABET=/opt/babet/bin/babet ./run_tests.sh
```

Complete minimal example

```
#!/usr/bin/env babet

local webdriver = require("webdriver")

local driver, err = webdriver.firefox({ headless = true })
if not driver then
    io.stderr:write(tostring(err), "\n")
    os.exit(1)
end
```

```

end

local ok, run_err = xpcall(function()
    assert(driver:open("https://example.com"))
    print(assert(driver:title()))

    local heading = assert(driver:css("h1"))
    print(assert(heading:text()))
end, debug.traceback)

local quit_ok, quit_err = driver:quit()
if not quit_ok then
    io.stderr:write("WebDriver shutdown: ", tostring(quit_err), "\n")
end

if not ok then
    io.stderr:write(tostring(run_err), "\n")
    os.exit(1)
end

```

Modules in a subdirectory

To keep the modules under lib/ instead:

```

my-project/
├── lib/
│   ├── webdriver.lua
│   ├── driver_manager.lua
│   └── webdriver_worker.lua
└── main.lua

```

Add the directory before the first require:

```

package.path = "./lib/?.lua;" .. package.path

local webdriver = require("webdriver")

```

Creating a session

Firefox

```

local driver, err = webdriver.firefox({
    headless = true,
})
assert(driver, err)

```

Chrome or Chromium

```

local chrome = assert(webdriver.chrome({ headless = true }))
local chromium = assert(webdriver.chromium({ headless = true }))

```

The convenience constructors never modify the caller's option table.

Generic form

```

local driver = assert(webdriver.start({
    browser = "firefox",
    headless = true,
}))

```

Start options

Browser and capabilities

Option	Type	Default	Description
browser	string	firefox	firefox, chrome, chromium
headless	boolean	false	adds the browser-specific headless arg
args	dense array	{}	browser command-line arguments
binary	string	automatic for Chromium	explicit browser executable
user_data_dir	string	absent	explicit Chrome/Chromium profile
window_size	{w,h}	absent	initial window size
accept_insecure_certs	boolean	false	corresponding W3C capability

```
local driver = assert(webdriver.firefox({
  headless = true,
  args = { "--private-window" },
  binary = "/usr/lib/firefox/firefox",
  window_size = { 1600, 1000 },
  accept_insecure_certs = false,
}))
```

For `webdriver.chromium()`, the executable is searched in `PATH` under the names `chromium` and then `chromium-browser`. For `webdriver.chrome()`, the library tries `google-chrome-stable`, `google-chrome` and then `chrome`, but lets `ChromeDriver` apply its own normal detection when none of those names is found.

Chromium distributed as a Snap

`ChromeDriver` normally creates a temporary profile. With Chromium installed as a Snap, that profile may be outside the filesystem space visible through Snap confinement, which can cause `DevToolsActivePort` file doesn't exist.

When the detected executable is under `/snap/bin/` or `/snap/chromium/`, the library therefore creates a temporary profile under:

```
~/snap/chromium/common/babet-webdriver/profiles/
```

That profile is passed through `--user-data-dir=...` and removed from `driver:quit()`. The library does not add `--no-sandbox`; the browser sandbox remains enabled.

An explicit profile is still supported:

```
local driver = assert(webdriver.chromium({
  headless = true,
  user_data_dir = os.getenv("HOME") .. "/snap/chromium/common/my-profile",
}))
```

`user_data_dir` cannot be combined with a manually supplied `--user-data-dir=...` argument in `args`. A profile supplied by the caller is never removed automatically.

External driver

Option	Type	Default	Description
driver_path	string	<code>PATH</code> lookup	explicit driver path
auto_install	boolean	true	installs the driver when missing
trust_on_first_use	boolean	false	permits the first unpinned artifact
expected_sha256	hexadecimal string	absent	expected archive hash
platform	string	automatic	forced platform key
cache	string	default cache	cache directory
force_driver_download	boolean	false	forces reinstallation

Option	Type	Default	Description
--------	------	---------	-------------

Port and connection

Option	Type	Default	Description
port	integer	automatic	WebDriver server port
port_attempts	integer	5	retries for automatic port selection
attach	boolean	false	does not start a process
start_timeout	seconds	15	startup time budget
status_timeout	seconds	1	timeout for each /status probe
poll_interval	seconds	0.1	delay between probes

Without an explicit port, the library asks the kernel for a free port. The temporary socket must then be closed before the driver starts, so a small race window remains. `port_attempts` lets the library retry when another process takes the port in that interval.

In attach mode, the historical defaults are preserved:

- Firefox: 4444;
- Chrome/Chromium: 9515.

```
local driver = assert(webdriver.firefox({
  attach = true,
  port = 4444,
}))
```

HTTP, screenshots and logs

Option	Default	Description
request_timeout	120 s	WebDriver command timeout
max_body_size	64 MiB	maximum HTTP response body
screenshot_max_size	64 MiB	limit after Base64 decoding
screenshot_permissions	0644	permissions of the published PNG
screenshot_durable	true	fsyncs the file and directory
log_path	automatic	exact log file
log_dir	cache/logs	automatic log directory
log_append	true	appends instead of truncating
log_permissions	0600	creation permissions
terminate_grace	2 s	grace period before SIGKILL

Options are strict. A typo such as `timeot = 5` immediately raises a Lua programming error.

Driver lifecycle

When the library starts the driver, it keeps the object returned by `babet.spawn()`:

```
print(driver:pid())
print(driver:port())
print(driver:log_path())
print(driver:is_running())
```

The streams are configured as follows:

- `stdin` is connected to `/dev/null`;
- `stdout` is redirected to the log file;
- `stderr` is merged into `stdout`.

No shell interprets the executable path or its arguments.

Closing

```
local ok, err = driver:quit()
assert(ok, err)
```

quit():

1. requests deletion of the W3C session;
2. sends SIGTERM to the driver process group;
3. waits for terminate_grace;
4. uses SIGKILL when necessary;
5. closes the Babet descriptors.

The method is idempotent. `driver:close()` is an alias. The object also exposes an `__close` metamethod for Lua to-be-closed variables.

Navigation

```
assert(driver:open("https://example.com"))
print(assert(driver:url()))
print(assert(driver:title()))
print(assert(driver:source()))
assert(driver:back())
assert(driver:forward())
assert(driver:refresh())
```

Locating elements and waiting

Strategies

```
driver:find("main h1") -- CSS
driver:find("//main//h1", { by = "xpath" })
driver:find("submit", { by = "id" })
driver:find("email", { by = "name" })
driver:find("button", { by = "tag" })
driver:find("active", { by = "class" })
driver:find("Home", { by = "link" })
driver:find("Hom", { by = "pLink" })
```

Convenience methods:

```
driver:css(selector)
driver:xpath(selector)
driver:id(value)
driver:name(value)
driver:tag(value)
```

id, name and class are converted to correctly escaped CSS selectors.

One element

```
local element, err = driver:css("h1")
assert(element, err)
```

When no element matches, the server returns the W3C no such element error.

Multiple elements

```
local elements = assert(driver:find_all("article"))
for _, element in ipairs(elements) do
```

```
print(assert(element:text()))
end
```

An empty array is a normal successful result.

Checking presence

```
local exists, err = driver:exists("#optional")
assert(exists ~= nil, err)
if exists then
    print("present")
end
```

Only no such element becomes false. A network failure, a closed session or a dead driver remains an error.

Waiting for a state

```
local button = assert(driver:wait("#submit", {
    state = "clickable",
    timeout = 20,
    interval = 0.2,
}))
```

States:

- present: the element is found;
- visible: found and displayed;
- clickable: displayed and enabled;
- gone: absence is confirmed.

By default, a stale reference causes a fresh lookup. Other errors are returned immediately.

Custom wait

```
local result = assert(driver:wait_until(function()
    local value, err = driver:js("return window.appReady === true")
    if value == nil and err then return nil, err end
    return value
end, 30, 0.25))
```

A callback exception or an error returned as the second value is not silently ignored.

Elements

Simple actions

```
assert(element:click())
assert(element:clear())
assert(element:type("Hello"))
assert(element:submit())
```

submit() finds the enclosing form through JavaScript and prefers requestSubmit() when available.

Reading values

```
print(assert(element:text()))
print(assert(element:tag()))
```

```

print(assert(element:rect()).width)
print(assert(element:css("display")))
print(assert(element:property("value")))
print(assert(element:dom_attr("data-id")))
print(assert(element:attr("textContent")))
print(assert(element:value()))

```

attr() mirrors Selenium's practical behavior: it tries the HTML attribute and then the DOM property.

Boolean states

```

local displayed, err = element:displayed()
assert(displayed ~= nil, err)

```

displayed, enabled and selected propagate errors. They no longer turn an error into false.

Nested lookup

```

local row = assert(driver:css("tr.active"))
local button = assert(row:find("button.save"))
local cells = assert(row:find_all("td"))

```

JavaScript

```

local text = assert(driver:js(
    "return arguments[0].textContent",
    element
))

```

Element objects passed as arguments are serialized with the W3C element key. Elements returned by the script are rebuilt automatically, including inside a nested table.

Cyclic tables are rejected before the request is sent.

Keyboard and mouse actions

```

assert(driver:actions()
    :move_to(element)
    :click()
    :send_keys("Hello")
    :perform())

```

Methods:

```

move_to(element [, x, y])
move_by(x, y)
click([element])
double_click([element])
context_click([element])
click_and_hold([element])
release()
key_down(character)
key_up(character)
send_keys(text)
pause(milliseconds)
drag_and_drop(source, destination)

```



```
perform()
clear()
```

The builder aligns keyboard and pointer sources tick by tick. `perform()` resets the sequences. Special keys are exposed through `webdriver.keys`, for example:

```
webdriver.keys.ENTER
webdriver.keys.CONTROL
webdriver.keys.DELETE
webdriver.keys.F1
```

Windows, tabs and frames

```
local current = assert(driver:window())
local handles = assert(driver:windows())
assert(driver:switch(handles[#handles]))
assert(driver:switch_last())
local new_handle = assert(driver:new_tab())
assert(driver:close_window())
```

Window rectangles:

```
assert(driver:set_window_rect({ x = 0, y = 0, width = 1280, height = 900 }))
local rect = assert(driver:window_rect())
```

Frames:

```
assert(driver:frame(0))
assert(driver:frame(frame_element))
assert(driver:parent_frame())
assert(driver:top_frame())
```

Alerts, cookies and timeouts

Alerts

```
print(assert(driver:alert_text()))
assert(driver:alert_send("answer"))
assert(driver:accept_alert())
assert(driver:dismiss_alert())
```

Cookies

```
local cookies = assert(driver:cookies())
assert(driver:set_cookie({ name = "theme", value = "dark" }))
assert(driver:delete_cookie("theme"))
assert(driver:clear_cookies())
```

A cookie name is encoded as a URL path segment; slashes, spaces and other characters cannot break the WebDriver path.

W3C timeouts

Values are expressed in seconds:

```
assert(driver:set_timeouts({
  implicit = 0,
  page_load = 60,
```

```
script = 30,  
}))
```

Screenshots

Without a path, the Base64 string is returned:

```
local encoded = assert(driver:screenshot())  
local element_encoded = assert(element:screenshot())
```

With a path, Babet decodes the binary data and publishes the file atomically:

```
assert(driver:screenshot("captures/page.png"))  
assert(element:screenshot("captures/button.png"))
```

The parent directory is created recursively when necessary. Atomic publication ensures that a reader sees either the previous complete file or the new one, never a partial PNG.

Automatic driver management

`driver_manager.lua` can also be used directly:

```
local manager = require("driver_manager")  
local path = assert(manager.install("firefox", {  
    trust_on_first_use = true,  
}))
```

Resolution

- geckodriver: latest official GitHub release, unless `driver_manager.gecko_version` is forced;
- chromedriver: version matching the installed Chrome/Chromium milestone, unless `driver_manager.chrome_version` is forced.

Verification

The cache stores one record per artifact:

```
~/.cache/babet-webdriver/pins/<driver>_<platform>_<version>.json
```

Each record contains:

- the URL;
- the archive SHA-256;
- the extracted executable SHA-256;
- the version and platform.

A cached executable is reused only when its hash matches. Legacy `pins.json` files containing only a hash string are read for migration, but the new format is written under `pins/`.

TOFU

Without a known pin, installation fails by default. Setting `trust_on_first_use = true` trusts the first HTTPS download and stores its hashes.

This mode detects later modification. It does not prove that the first artifact was legitimate. For a stronger trust chain, supply `expected_sha256` from an independent channel.

Extraction

The archive is never loaded entirely into Lua. Babet downloads it to a temporary file, verifies the hash, and then extracts only:

- geckodriver for Firefox;
- chromedriver-<platform>/chromedriver for Chrome.

Anti-bomb limits apply to the complete archive.

Workers and channels

webdriver_worker.lua keeps the session and driver process inside a persistent worker.

```
local worker_driver = require("webdriver_worker")
local session = assert(worker_driver.start({
    browser = "firefox",
    headless = true,
    channel_capacity = 64,
    command_timeout = 120,
}))

assert(session:open("https://example.com"))
local heading = assert(session:css("h1"))
print(assert(heading:text()))
assert(session:stop())
```

Before creating the worker, the parent prepares the driver when necessary. Multiple workers therefore do not download the same artifact concurrently.

Element proxies

A WebDriver element is not sent as userdata. The worker sends its W3C id and the parent creates a proxy:

```
local element = assert(session:css("h1"))
assert(worker_driver.is_element(element))
print(element:element_id())
```

Proxies can be reused as arguments to `js()` or `frame()`. The internal transport also preserves nil arguments, including in the middle of an argument list, multiple return values, and the W3C error code.

Lifecycle

```
print(session:status())
assert(session:stop(10))
```

`stop()` first asks the worker to close the session cleanly. On timeout, Babet cancellation is requested. Cancellation remains cooperative, so every HTTP operation in the module is bounded.

A proxy session is synchronous and must have only one command in flight. Create multiple sessions for parallel execution.

The chainable `actions()` builder is not transported through the worker proxy. Chainable actions remain available on a direct session.

Error contract

An operational error returns:

```
nil, "webdriver: ..."
```

A WebDriver response preserves its W3C code in the message and, for methods that expose it such as `find()`, as a third return value:

```
local element, err, code = driver:find("#missing")
assert(element == nil and code == "no such element")
```

Example messages:

```
webdriver: no such element: element missing
webdriver: stale element reference: ...
webdriver: invalid session id: ...
```

Programming errors that raise a Lua exception include:

- unknown option;
- invalid type;
- sparse argument array;
- unknown locator strategy;
- invalid frame type;
- cyclic table passed to JavaScript.

Limitations

- synchronous protocol;
- automatic driver management currently supports Linux only;
- Chrome for Testing provides a Linux x86-64 binary only;
- no WebDriver BiDi;
- no authenticated or TLS remote proxy for the WebDriver server;
- no download resume;
- no progress callback;
- worker proxy limited to serializable methods and without the Actions builder;
- Babet channels carry JSON-like data, not binary strings containing NUL bytes.

WebDriver session in a worker

The complete integration path can be verified with:

```
HEADLESS=1 ./run_worker_smoke.sh firefox
HEADLESS=1 ./run_worker_smoke.sh chromium
```

This test covers the parent process, channels, worker, driver process and real browser.